応用情報技術者試験　テクノロジ系　#2

アルゴリズムとプログラミング

用語集まとめ ― 過去問10回分（令和3〜7年度）で出たところを中心に



全体像 シラバスの5つの小分類で読む

1 データ構造
配列・リスト・スタック・キュー・木（2分探索木・AVL木・ヒープ）・ハッシュ表

2 アルゴリズム
流れ図・整列・探索・再帰・設計方法・グラフ・文字列照合・記憶域管理

3 プログラミング
プログラムの性質（再入可能など）・構造・データ型・Web・BNF

4 プログラム言語
言語の分類と特徴・型付け・引数の渡し方・記憶域

5 その他の言語
HTML・XML・JSON・スタイルシート・UML

午前での出方	
問題数	毎回 **問5〜問7 の3問**（R3春だけ問5〜問8 の4問）。10回で31問
同じ問題の再出題	3組（スタックの出力順、ハッシュの衝突条件、R言語）
問題の型	手を動かして追う問題（並べ替えの途中・木の操作・リストのポインタ）が約半分。残りは「説明を選ぶ」で、誤りの選択肢は同じ仲間（ソートなら4種類）

この資料の読み方：1枚に1つのメッセージ。見出しの一文と、図の動きを自分の手で追えれば解けるようにしてある。

過去10回・31問の内訳 令和3年度春期〜令和7年度秋期

テーマ	数	よく出る論点
リスト・スタック・ハッシュ	6	スタックの出力順（2回）、配列で作るリスト、双方向リストの挿入、ハッシュの衝突（2回）
整列（ソート）	5	バブルソート（4回）、クイックソートの途中経過
木	4	配列で表す2分木、2分探索木の削除、AVL木の回転、なぞり方
プログラム言語	5	R（2回）、Python、TypeScript、Scala
再帰・設計・流れ図	3	分割統治法、階乗の再帰、ループの終了条件
探索の比較回数	2	線形探索の平均、ブロックに分けた探索
記憶域の管理	2	ベストフィット（2回）
プログラムの性質	2	再入可能・再使用可能・再帰的・再配置可能
XML・JSON	2	符号化宣言の省略、JSON

一番の得点源：バブルソート。10回で4回、説明文・流れ図・途中経過と形を変えて出た。

手で追う問題：スタック・リスト・木は、紙に図を描いて1手ずつ動かせば必ず解ける。

落とし穴：言語の問題は誤りの選択肢も実在の言語（Go・Kotlin・JavaScript など）。「型付け」と「何向けか」で見分ける。

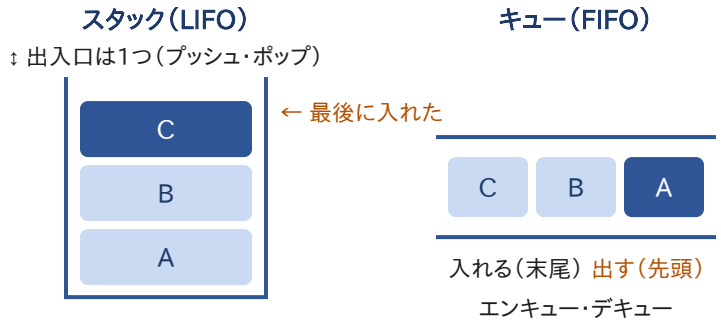
第1章

データ構造

配列とリスト ／ スタックとキュー ／ 木（2分探索木・AVL木・ヒープ）／ ハッシュ表



スタックは「後から入れたものが先」、キューは「先に入れたものが先」



使い道：スタック＝関数の戻り先・逆ポーランド記法・深さ優先探索。キュー＝待ち行列・幅優先探索。優先度の高いものから出すのは**優先度付きキュー**（ヒープで作る）。

R3春 問5＝R7春 問5：A, B, C の順に入れ、出し入れ1回ずつ

出力順	できるか	操作（入＝プッシュ、出＝ポップ）
A B C	○	A入出 → B入出 → C入出
A C B	○	A入出 → B入 → C入出 → B出
B A C	○	A入 → B入出 → A出 → C入出
B C A	○	A入 → B入出 → C入出 → A出
C B A	○	A入 → B入 → C入出 → B出 → A出
C A B	✕	C を先に出すと、一番上は B。A を B より先に出せない

並べ方6通りのうち、作れないのは CAB だけ → **5通り（ウ）**。

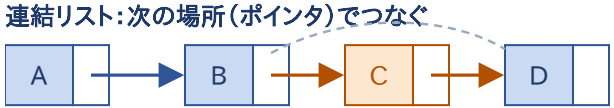
🔗 配列は「参照が速い」、連結リストは「挿入・削除が速い」

配列：連続した領域に並べる（添字で直接）



使わない領域（最大個数分を確保）

連結リスト：次の場所（ポインタ）でつなぐ



付け替えは2か所

双方向リストは前へのポインタももつ。挿入では、新しい要素の次・前と、前後の要素の計4か所を書き換える。

操作	配列で作る	ポインタで作る
i番目を見る	**すぐ**（添字）	先頭からたどる（要素数に比例）
途中に挿入・削除	後ろをずらす（要素数に比例）	**付け替えだけ**（一定時間）
領域	最大個数分を先に確保。**使わない領域が出る**	要素の分だけ。ポインタの領域が余計に要る

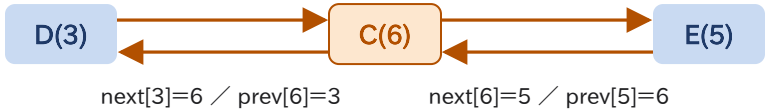
R4春 問5：配列で作ったリストの特徴は「最大個数分の領域を確保し、使われない領域が出る（ア）」。イ・ウ・エはどれもポインタの方の特徴。

🔗 配列で作る双方向リストは「Headから next をたどって並びを復元」

R5秋 問5：D の次に C を挿入（追加は配列の末尾＝6番）

要素番号	1	2	3	4	5	6
elem	A	F	D	B	E	C
next	4	0	5 → 6	3	2	5
prev	0	5	4	1	3 → 6	3

Head＝1 から next をたどる：A(1) → B(4) → D(3) → E(5) → F(2)。
next が0の F が末尾（Tail＝2）。

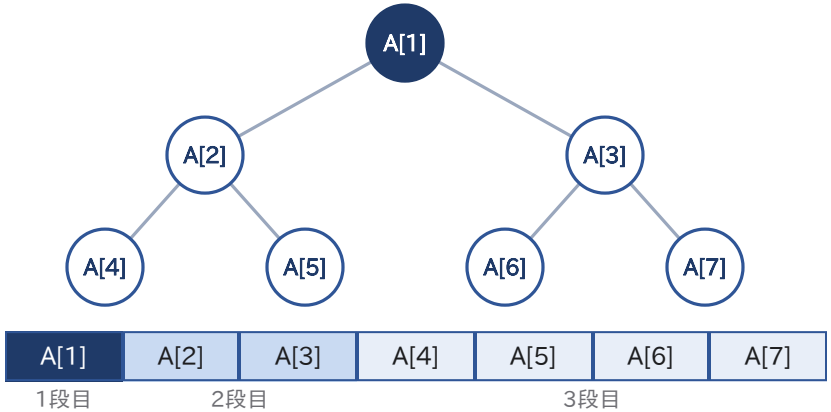


書き換えは4か所。問われたのは C の2つ

答えは next[6]=5、prev[6]=3（ウ）。誤りの選択肢は、F(2)・B(4) など並びを取り違えた番号。

コツ：配列の番号順（A, F, D, B, E）はリストの順ではない。必ず Head から next をたどって並びを書き出す。

🌳 完全2分木は配列で表せる。子は 2i と 2i+1



木の部品

根＝一番上（親なし）、**葉**＝子のない節、**枝**＝親子を結ぶ線（節が n 個なら n−1 本）、**部分木**＝ある節から下の小さな木

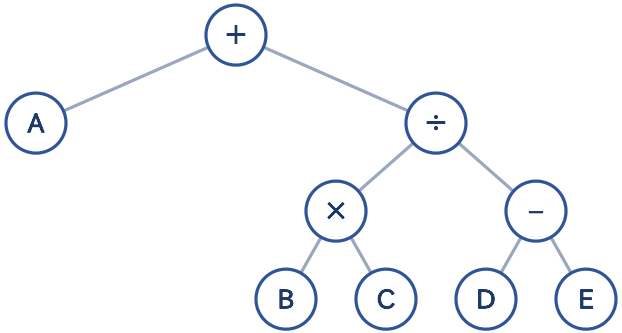
完全2分木

一番下の段以外は埋まり、一番下の段は左から詰める。A[i] の子は A[2i]・A[2i+1]、親は A[i÷2]（切捨て）

R3春 問6：この表し方で配列を先頭から調べると、1段目→2段目→3段目と**段ごとに左から右**へ進む。これは**幅優先探索（エ）**。行きがけ順なら A[1]→A[2]→A[4] と下へ潜るので順番が違う。

🌳 木のなぞり方は「節をいつ出力するか」の違い

R6春 問6 の木（数式を表す2分木）



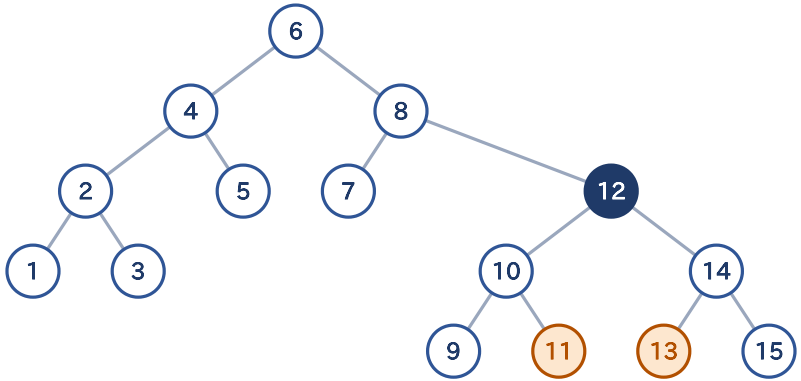
深さ優先でなぞるとき、節を**先に**出すか、左右の**間**か、**後**か。左右のどちらから行くかも問題文で必ず確かめる。

なぞり方	出力
先行順（行きがけ） 自分→左→右	＋A÷×BC-DE 前置記法
中間順（通りがけ） 左→自分→右	A＋B×C÷D-E 中置記法（かっこは省略）
後行順（帰りがけ） 左→右→自分	ABC×DE-÷＋ 逆ポーランド記法＝R6春 問6 のイ
R6春 問6 右→左→自分	**ED-CB×÷A＋（エ）**

この問題は「右の子を先」に処理する。普通の後行順（イ）を選ばせる作りなので、定義の1行目を読み飛ばさない。

2分探索木は「左＜節＜右」。消したら左の最大か右の最小で埋める

R6秋 問5:12 を削除して、別の要素を移すだけで直す



2分探索木の決まり

どの節でも、左の部分木は全部小さく、右の部分木は全部大きい。**中間順になぞると小さい順**に並ぶ

削除のしかた

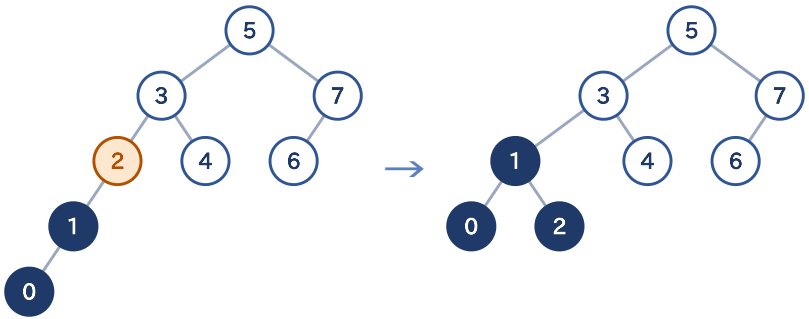
葉なら消すだけ。子が1つならその子を上げる。子が2つなら**左の部分木の最大値**か**右の部分木の最小値**を移す

12 の左の部分木の最大は 11、右の部分木の最小は 13。選択肢にあるのは **13（ウ）**。9・10・14 を置くと、残った節で大小の決まりが崩れる。

偏った形（片側に一直線）になると探索が O(n) に落ちる。これを防ぐのが次の AVL木。

AVL木は「左右の高さの差が2になった一番下の節」で回転する

R7春 問6：5(3(2,4),7(6)) に 1 と 0 を順に追加



0 を足すと節2で左の高さ2・右0　　節2を右回転して 1 を上げる

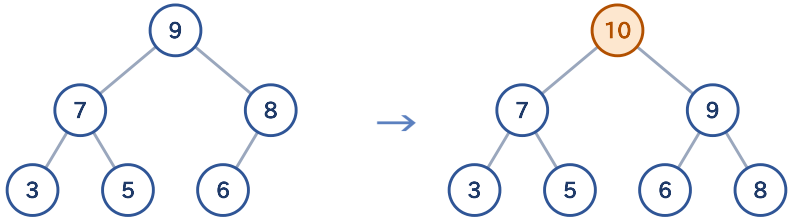
AVL木
どの節でも左右の部分木の高さの差が1以下の2分探索木。崩れたら**回転**で直す

右回転（左に偏ったとき）
偏った節の左の子を上に上げ、偏った節はその右の子になる。3より上と右側（7・6）は動かさない

答えは **5(3(1(0,2),4),7(6))（ウ）**。ほかの選択肢も「高さの差1以下」は満たすが、根まで組み替えたり（ア・イ）、偏っていない右側を回したり（エ）していて、追加の手順どおりではない。

ヒープは「親≧子」の完全2分木。根がいつも最大

最大ヒープに 10 を追加する



配列：9, 7, 8, 3, 5, 6　　配列：10, 7, 9, 3, 5, 6, 8

10 を末尾（8 の右の子）に置き、親より大きければ交換：8 と交換 → 9 と交換 → 根へ。高さの分だけなので O(log n)。

操作	やり方
追加	末尾に置いて、親と比べながら上へ
根を取り出す	最後の葉を根へ移し、大きい方の子と比べながら下へ
ヒープソート	根（最大）を取り出して末尾へ、を繰り返す。O(n log n)

2分探索木との違い：ヒープは左右の子の大小を決めない。決まっているのは親子の大小だけ。IPA の問題文では「順序木」と書かれることもある（R3秋 問5 エ）。

プログラムの「ヒープ領域」（実行中に確保する記憶域）とは別物。

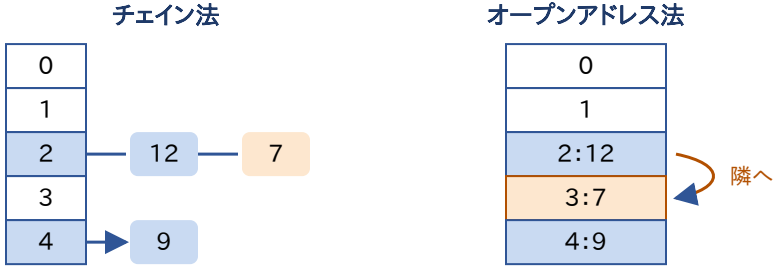
ハッシュは「余りが同じなら衝突」。差が n の倍数のとき

h(x)=x mod 5（表の大きさ n=5）		
キー	÷5の余り	格納位置
12	2	2
7	2	2 ← 衝突（シノニム）
9	4	4

R4秋 問5=R6秋 問6：a mod n = b mod n のとき衝突。余りが同じ ⇔ **a−b が n の倍数（イ）**。12−7=5 は5の倍数。

「n が a−b の倍数」（エ）は向きが逆。日本語の「AがBの倍数」の向きで迷わせる選択肢。

衝突したときの2つの対策



チェイン法＝同じ位置から**リストでつなぐ**。オープンアドレス法＝表の中の**別の空き**を探す。衝突が少なければ探索は平均 O(1)。

第2章

アルゴリズム

流れ図 ／ 整列 ／ 探索 ／ 再帰と設計方法 ／ グラフ・文字列 ／ 記憶域の管理



流れ図は記号の形で役割が決まる。ループは「いつ抜けるか」を数で確かめる



ループ端には「変数名：初期値，増分，終値」を書く。**定義済み処理**は両端に縦線（中身は別の流れ図）。

R6春 問5：x←1, n←1 から「x←x×n, n←n+1」を a まで繰り返す

回	x←x×n	n←n+1	a＝「n＞M」（M=3）
1	1×1=1	2	No
2	1×2=2	3	No
3	2×3=6	4	**Yes → 終了**

左の流れ図は M から1まで掛けて M!=6。右も 6 で止めるには、M まで掛けた直後（n=M+1）で抜ける **n＞M（ウ）**。n＞M−1 だと1回足りず、n＞M+1 だと1回多い。

コツ：M=3 のような小さい数を入れて、表で1行ずつ追う。

バブルソートは「1回なぞるごとに、端に最大が1つ決まる」

R5秋 問6：状態の移り変わり（濃い所が決まった値）						
状態0	3	5	9	6	1	2
状態1	3	5	6	1	2	**9**
状態2	3	5	1	2	**6**	**9**
状態3	3	1	2	**5**	**6**	**9**
状態4	1	2	**3**	**5**	**6**	**9**

隣どうしを比べて逆なら交換、を左から右へ。9 が泡のように右端へ浮かび、次は 6、次は 5。**決まっていない部分（3, 5, 1, 2）の並びは大きく崩れない**のが見分けどころ → **バブルソート（ウ）**。

誤りの選択肢	この状態にならない理由
クイックソート	基準値（例えば3）で分けるので、状態1で 1, 2 が前へ移るはず
挿入ソート	先頭から整列済みの部分が伸びる。末尾から決まらない
ヒープソート	最初にヒープを作るので、全体の並びが大きく入れ替わる

バブルソートは10回で4回出た：説明文（R3秋 問5・R6春 問7）、流れ図（R4秋 問6）、途中経過（R5秋 問6）。比べる向きが「末尾から先頭へ」でも、隣どうしの交換ならバブルソート。

ソートの説明文は「決まり文句」で見分ける

方法	選択肢に出る決まり文句	計算量（平均／最悪）	ひとこと
バブル	隣り合う要素を比較して、逆なら入れ替える	$O(n^2)$／$O(n^2)$	安定。端から1つずつ決まる
選択	未整列の中から最小値を探索し、未整列の先頭と入れ替える	$O(n^2)$／$O(n^2)$	交換は1回のなぞりで1回
挿入	整列済みの部分の適切な位置に挿入する	$O(n^2)$／$O(n^2)$	ほぼ整列済みなら速い
シェル	ある間隔おきに取り出した部分列を整列し、間隔を詰める	間隔による	挿入ソートの改良
クイック	基準値より大きい区分と小さい区分に振り分ける	$O(n \log n)$／$O(n^2)$	実用上は速い。分割統治法
マージ	ほぼ同じ長さの2つに分割し、分割できなくなったら併合しながら並べる	$O(n \log n)$／$O(n \log n)$	安定。作業領域が要る
ヒープ	未整列の部分を順序木（ヒープ）にし、最大（最小）を取り出して移す	$O(n \log n)$／$O(n \log n)$	作業領域がほぼ不要

R3秋 問5（バブルの説明）の誤りはシェル・クイック・ヒープ、**R6春 問7**の誤りはクイック・マージ・選択。この表の7つから4つを並べるのが定番。**安定**＝同じキーの要素の順番が整列の前後で変わらない（バブル・挿入・マージ）。

流れ図のソートは内側のループ、クイックソートは「基準値で2つに分ける」

R4秋 問6 の流れ図（文字で書き直した）

```
ループ1　i：1，1，n-1
  ループ2　j：n，-1，i＋1
    A[j]＜A[j-1] なら A[j] と A[j-1] を交換
  ループ2 の終わり
ループ1 の終わり
```

内側で**隣どうし（j と j−1）**を比べて交換している → **バブルソート（エ）**。末尾から先頭へ進むので、1回ごとに最小値が A[i] に決まる。選択ソートなら「最小値の位置を覚えて最後に1回交換」になる。

R5春 問7：2, 3, 5, 4, 1 をクイックソート（基準値は左端）

分割	基準値	小さいグループ	大きいグループ	並び
1回目	2	1	3, 5, 4	1, 2, 3, 5, 4
2回目	3	なし	5, 4	**1, 2, 3, 5, 4（ア）**
3回目	5	4	なし	1, 2, 3, 4, 5

「グループ内の順番は元の配列と同じ」なので、大きいグループは 3, 5, 4 のまま。イ（5, 4, 3）はこの条件に反し、ウ・エは 1 が 2 の後ろにあって分割になっていない。

探索の比較回数は「見つかる場合×確率＋見つからない場合×確率」

方法	平均	最大
線形探索	(n+1)/2（見つかる場合）	n
2分探索（要整列）	約 $\log_2 n$	$\lfloor \log_2 n \rfloor + 1$
ハッシュ表探索	衝突がなければ1	衝突しだい

2分探索は1回で範囲が半分。1,000個でも最大 **10回**（$2^{10}=1{,}024$）。線形探索は番兵（探す値を末尾に置く）で「表の終わりか」の判定を省ける。

R5春 問6：線形探索、表にない確率が a

$$(n+1)/2 \times (1-a) + n \times a$$

見つかる場合（平均）　ない場合（最後まで n 回）

R7秋 問6：m 個ずつのブロックに分けて探す

$$(n/m)/2 + m/2 = n/(2m) + m/2$$

ブロックを探す（平均）　ブロックの中を探す（平均）

誤りの選択肢は「片方だけ」の式（ブロックを探す回数だけ、見つかる場合だけ）や「最大の回数」。2段階の両方を足しているかを確かめる。

再帰は「止まる条件」と「小さくして自分を呼ぶ」の2つでできている

R7春 問7：fact(n) の再帰的な定義

```
if n＝0 then return 1
else return n × fact(n−1)
```

fact(3)＝3×fact(2)　　3×2＝6

fact(2)＝2×fact(1)　　2×1＝2

fact(1)＝1×fact(0)　　1×1＝1

fact(0)＝1（止まる）　　戻りながら掛ける

選択肢	どうなるか
0 を返す, n−1	最後に0が掛かり、いつも0
0 を返す, n+1	n が増え続けて止まらない
1 を返す, n−1	**正しい（ウ）**
1 を返す, n+1	n＝0 にたどり着かず止まらない

呼ぶたびに引数と戻り先が**スタック**に積まれる。止まる条件がないと積み続けてあふれる（スタックオーバーフロー）。だから再帰には、呼出しごとに別のデータ領域が必要（→第3章「再帰的」）。

🧩 設計方法は「分けて解く・その場で最良・表に記録・見込みで切る」

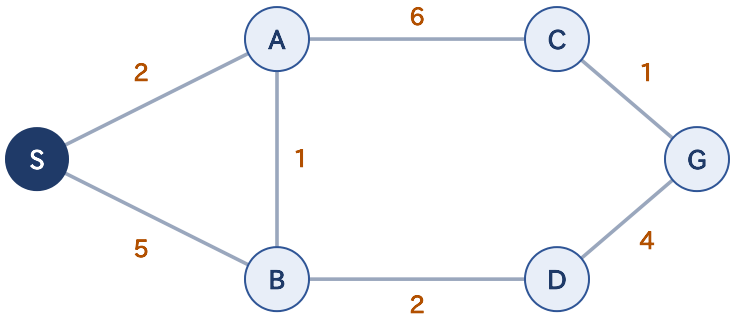
方法	考え方	代表例	R3春 問7 では
分割統治法	小さな問題に分け、独立に解いた結果をつなぎ合わせる	マージソート、クイックソート、2分探索	**ウ（正解）**
貪欲法	その時点で最良のものを選び続ける。全体の最適とは限らない	ダイクストラ法、おつりの硬貨	エ（各時点で最良の解を選択）
動的計画法	部分問題の答えを表に記録して使い回す	ナップサック問題、最短経路	—
分枝限定法	場合分けしながら、見込みのない枝を調べない	巡回セールスマン問題	イ（無駄な組合せを除く）に近い
バックトラック法	深さ優先で進み、だめなら1つ前に戻る	迷路、8クイーン	—
逐次改良	粗い解から少しずつよくしていく	山登り法	ア（粗い解を逐次改良）

確率アルゴリズム：モンテカルロ法＝乱数で何度も試し、割合から近似（円の中に入った点の割合で円周率）。

遺伝的アルゴリズム：解を遺伝子に見立て、選択・交叉・突然変異。最適に近い解を探す近似の方法。



🌐 ダイクストラ法は「一番近い未確定の点を確定」を繰り返す



手順	確定する点	S からの距離（未確定は暫定）
1	S=0	A=2, B=5
2	A=2	B=min(5, 2+1)=3, C=2+6=8
3	B=3	D=3+2=5
4	D=5	G=5+4=9
5	C=8	G=min(9, 8+1)=9
6	G=9	S→A→B→D→G（または S→A→C→G）

重みが負の辺があると使えない（確定した距離があとで縮むため）。負の辺があるときは**ベルマンフォード法**（全部の辺で更新を繰り返す）。辺の重みがなければ**幅優先探索**で辺の本数が最少の道が求まる。

文字列照合は「どこから比べ、どれだけずらすか」の違い

本文「ABCABDABCABC」からパターン「ABCABC」を探す

方法	比べ方											
本文	A	B	C	A	B	D	A	B	C	A	B	C
位置0	A	B	C	A	B	C						
力まかせ		A	…									
KMP				A	B	C	…					
BM							A	B	C	A	B	C

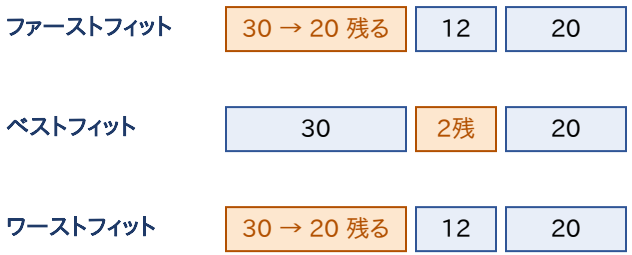
位置0は6文字目（本文の D）で不一致。**力まかせ**は1つずらして先頭から。**KMP**は一致した ABCAB の末尾 AB がパターンの先頭と同じことを使い、3つずらして D の所から比べ直す。**BM**は末尾から比べるので最初に D を見る。D はパターンにない文字なので**6つ一気にずらせる**。

方法	比べる向き	不一致のとき
力まかせ法	先頭から	1文字ずらして先頭から比べ直す
KMP法	先頭から	一致していた部分のパターン自身の情報で、比べ直す位置を決める。本文の中で後戻りしない
BM法	**末尾から**	不一致の文字に応じてパターンを大きくずらす。実際の文章では速い

見分ける一言：「末尾から」ならBM法、「パターン自身の情報（一致した部分）」ならKMP法。名前はどちらも考案者の頭文字。

📦 空き領域の割当て：ベストフィットは「入る中で一番小さい」

空き領域 30・12・20（アドレス順）に、10 を割り当てる



ベストフィットは 12 を使い、**2 という使いにくい小さな空き**が残る。これが積み重なるのが弱点（R7秋 問5 イ）。

方式	選ぶ領域	傾向
ファーストフィット	先頭から見て最初に入る	下位アドレスが細かくなる（R7秋 問5 ア）
ベストフィット	入る中で最小	小さな空きが多数残る
ワーストフィット	最大	大きな空きが減る

R5春 問5：ベストフィットを速くするには、空き領域を**大きさをキーにした2分探索木**（ウ）で持つ。大きさ順のリストでは先頭からたどるので O(n)。

細切れ（**フラグメンテーション**）は、使用中の領域を詰める**コンパクション**で解消する。

📜 言葉・圧縮・3次元図形のアルゴリズムは「何を順に決めるか」で覚える

形態素解析
単語に区切り品詞を決める
→
構文解析
係り受け（どこにかかるか）
→
意味解析
単語・文の意味を決める
→
文脈解析
指示語・省略を文をまたいで補う

圧縮	やり方
ランレングス法	AAAABBB → A4B3。同じ値が続くデータ（白黒画像・FAX）向き
ハフマン法	よく出る記号ほど短い符号。区切りなしで元に戻せる

n-gram：「東京都」→「東京」「京都」と n 文字ずつ区切って索引を作る。
文章間類似度は文章をベクトルにして近さを測る。

3次元図形	やり方
Zバッファ法	画素ごとに奥行き（Z値）を記録し、手前の物だけ描く
スキャンライン法	横1行（走査線）ごとに、手前に見える面を決める
レイトレーシング法	視点から光線を逆にたどり、反射・屈折・影を計算

Zバッファ法とスキャンライン法は**隠面消去**（隠れた面を消す）の方法。レイトレーシング法は写実的だが計算が重い。

第3章

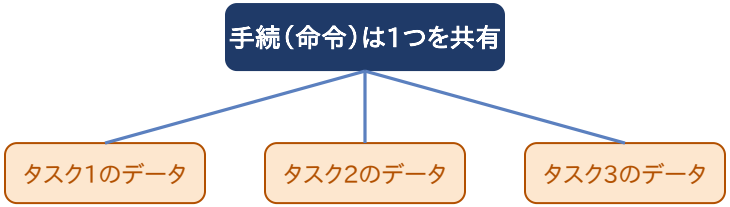
プログラミング

プログラムの4つの性質 ／ プログラムの構造 ／ データ型 ／ Webプログラミング ／ BNF



⚙ プログラムの性質は4つ。「再入可能＝同時に呼ばれても正しい」

性質	意味	そのための条件
再入可能 リエントラント	複数のタスクから同時に呼ばれても、それぞれ正しい結果	手続は共有、**データはタスクごと**
再使用可能 リユーザブル	読み込み直さずに、もう一度実行できる	始めに初期化する（終わりに初期状態へ）
再帰的 リカーシブ	自分自身を呼び出せる	呼出しごとに別のデータ領域
再配置可能 リロケータブル	主記憶のどこに置いても動く	アドレスを相対で表す



R4春 問6：再入可能の特徴は「並行して実行されても、それぞれのタスクに正しい結果を返す（エ）」。ア＝再配置可能、イ＝再帰的、ウ＝オーバーレイ。

R3秋 問6：正しいのは再使用可能の説明（イ）。「再入可能はデータとコードの**両方を共用**」「再帰的は**全てがデータを共用**」「再配置可能は**定まった領域**で実行」がそれぞれ誤り。

🛠️ オーバーロードは「同じ名前・違う引数」、オーバーライドは「親を書き直す」

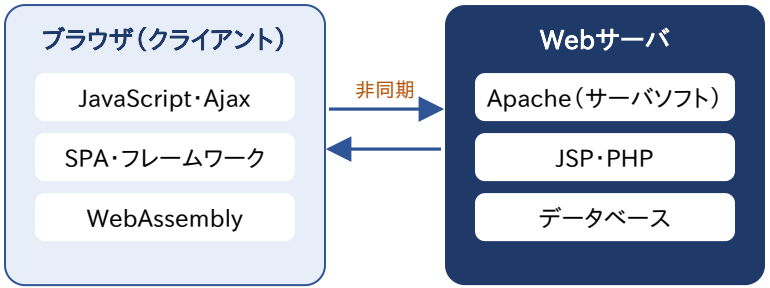
用語	意味
オーバーロード	同じクラスに、名前が同じで**引数の型や数が違う**メソッドを複数定義（多重定義）
オーバーライド	親クラスのメソッドを、**名前も引数も同じ**まま子クラスで書き直す
多重継承	複数の親クラスから引き継ぐ。C++はできる、Java・Scalaはクラスではできない
DLL	実行時に結び付ける部品の集まり。部品だけ差し替えられる
静的リンク	作るときに部品を実行ファイルに全部取り込む

```
class 図形 { 面積() }
class 円 extends 図形 { 面積() ← オーバーライド }

印刷(int x)  印刷(String s)  印刷(int x, int y)
↑ 同じ名前で引数が違う ＝ オーバーロード
```

モジュール分割：機能ごとに分け、モジュールの中のまとまりは強く、モジュールどうしのつながりは弱くするほど**独立性が高い**。**コーディング標準**では命名規則・字下げ・ネストの深さ・使用禁止命令（goto など）を決める。

🌐 Web技術は「ブラウザで動くか、サーバで動くか」で分ける



技術	見分ける一言
Ajax	画面を切り替えず、裏で**非同期に通信して一部だけ書き換える**
SPA	**1ページだけ**読み込み、中身を入れ替えて画面を変える
WebAssembly	C言語などから作った**バイナリをブラウザで高速**に実行
JSP	HTMLに**Java**を埋め込み、**サーバ**でページを作る
リッチクライアント	Webで配布しつつ、デスクトップアプリ並みの操作性

📋 BNFは「::=」と「|」だけ。繰返しは自分自身を使って書く

〈識別子〉::=〈英字〉|〈識別子〉〈英字〉|〈識別子〉〈数字〉
〈英字〉::=A|B|C
〈数字〉::=0|1|2

文字列	〈識別子〉か	理由
A1B	○	A → A1（＋数字）→ A1B（＋英字）
1AB	×	必ず〈英字〉から始まる
AB3	×	3 は〈数字〉に入っていない

記法	特徴
BNF	「::=」（定義）と「|」（または）。繰返しは再帰で表す
EBNF	BNFに { }（繰返し）や [ ]（省略可）を足して短く書ける
構文図	同じ決まりを、矢印でつないだ箱の図で表す

BNFで書けるのは**文脈自由文法**。多くのプログラム言語の**構文規則**（書き方の決まり）はこれで定める。何を意味するかの決まりは**意味規則**。

基礎理論の「形式言語とオートマトン」とつながる：文脈自由文法を受理するのはプッシュダウンオートマトン。

第4章

プログラム言語

言語の分類 ／ 過去問に出た言語 ／ 型付け ／ 引数の渡し方 ／ 記憶域



⌨ 言語は「水準」と「書き方（パラダイム）」の2軸で分ける

高水準言語
人の考え方に近い。C・Java・Python など

↓ コンパイラ・インタプリタ

中間言語
CPUに依存しない形（Javaのバイトコード、CLIの共通中間言語）

↓ 仮想マシン・JITコンパイラ

アセンブラ言語
機械語と1対1の記号（ADD・MOV）

機械語
CPUが直接実行する0と1

分類	考え方	例
手続型	手順を順に書き、変数を書き換えて進める	C、Fortran、COBOL
関数型	関数の定義と適用を組み合わせる。値を書き換えない	Haskell、Lisp
論理型	事実と規則を書き、推論で答えを出す	Prolog
オブジェクト指向	データと手続をまとめたオブジェクトが単位	Java、C++
スクリプト	手軽に書いてすぐ実行。動的型付けが多い	Python、PHP、Ruby、Perl

共通言語基盤（CLI）：違う言語で書いた部品を共通の中間言語にして同じ環境で動かす仕様（JIS X 3016）。.NET がこれに沿う。

💻 過去問の言語は「何向けか」と「型付け」で見分ける

言語	特徴	型付け	過去問
R	統計解析・機械学習。パッケージが多いOSS	動的	R3春 問8＝R6秋 問7
Python	**字下げ**でブロックを表す。AI・分析で人気	動的	R4春 問7
TypeScript	JavaScriptに**静的な型**を足した。JSに変換して実行	静的	R3秋 問7
Scala	**オブジェクト指向＋関数型**。JVMで動く	静的	R7秋 問7
Go	グーグル社。並行処理が得意	静的	R3春 問8 の誤り
Kotlin	Android開発。Javaと一緒に使える	静的	
ECMAScript JavaScript	JavaScriptの標準仕様と、それに沿った言語	動的	R3秋 問7 の誤り
VBScript	Visual Basicを元にしたMicrosoftの言語	動的	

静的型付け：実行前（コンパイル時）に型を決めて誤りを見つける。**動的型付け**：代入された値で実行時に型が決まる。**型推論**：型を書かなくても処理系が決める（Scala・Kotlin・Go・TypeScript）。

Scala（R7秋 問7）：クラスの多重継承はできない（トレイトで代用）。基本はJVMのバイトコードにコンパイルする。静的型付け。誤りの選択肢3つはこれを逆にしたもの。

ブロックの表し方：C・Java・PHP は { }、Pascal は begin〜end、Python は字下げ。

🔢 値呼出しはコピーを渡す、参照呼出しは場所を渡す

```
a←1, b←1
sub(a, b)  ← a, b は実引数
sub(x, y) { x←x＋10; y←y＋10 }  ← x, y は仮引数
```

渡し方	渡すもの	sub のあと
値呼出し(x)	a の値のコピー	a=1(変わらない)
参照呼出し(y)	b の場所(アドレス)	b=11(変わる)

プログラム領域	命令。実行中は変えない
静的領域	静的変数・大域変数・定数。ずっと残る
スタック領域	自動変数・戻り先。呼出しで確保、戻ると解放
ヒープ領域	実行中に必要な分だけ確保(new・malloc)

再帰で呼出しごとに別になるのは自動変数(スタック領域)。ヒープ領域の使われなくなったデータを自動で回収するのが**ガベージコレクション**。解放し忘れて減っていくのは**メモリリーク**。

第5章

その他の言語

HTML・XML ／ DTDとXML Schema ／ DOM・SAX ／ JSON ／ スタイルシート ／ UML



XMLは「タグを自分で決められる」。UTF-8とUTF-16なら符号化宣言を省略できる

SGML
マークアップ言語を定義するための元の規格

HTML
決まったタグだけ

XML
タグを自分で定義

XHTML
HTMLをXMLの規則で定義し直した。終了タグを省略できない

```
<?xml version="1.0" encoding="Shift_JIS"?>
<本><題名>応用情報</題名></本>
```

R4秋 問7: 符号化宣言（encoding）を省略できるのは **UTF-16（エ）**（とUTF-8）。XMLの処理系はこの2つを必ず読めることになっていて、先頭のバイトから見分けられる。EUC-JP・ISO-2022-JP・Shift_JIS は宣言が必要。

決まりの書き方	特徴
DTD	使える要素・属性を定義。SGMLからの独自の書き方。データ型は細かく決められない
XML Schema	同じ役割を**XMLの書き方**で。整数・範囲など**データ型**まで決められる

JSONは「名前と値の組」と「値の並び」の2つの形だけ

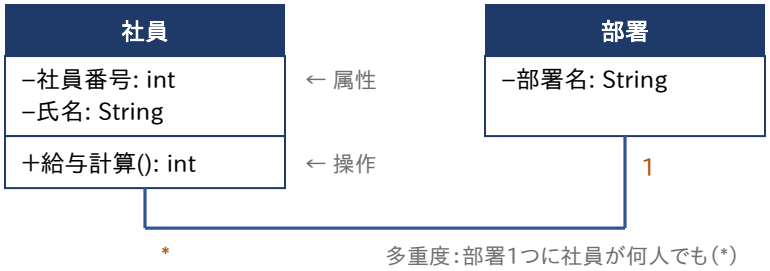
```
{
  "試験": "応用情報",   ← 名前と値の組
  "分野": ["基礎理論", "アルゴリズム"]   ← 値の順序付きリスト
}
```

R5秋 問7: JavaScriptのオブジェクトの表記法が元で、この2つの構造でデータを表すのは **JSON（イ）**。Web API のデータ交換で一番使われる。字下げで書く **YAML** は設定ファイル向き。

名前	役割
DOM	文書を**木の形**で全部読み込み、自由に読み書きする
SAX	先頭から読み、**タグが現れるたびに**処理を呼ぶ。メモリが少なくて済む
SOAP	XMLのメッセージで**ほかのシステムの機能を呼ぶ**通信の決まり
XSL	XMLを**別の形式に変換**したり見た目を決めたりする
CSS	構造は変えず**見た目だけ**を指定する
SVG	図形を線や形の情報として**XMLで書く**画像形式

R5秋 問7 の誤りは DOM・SOAP・XML。どれもXMLに関係するが、「データ記述の仕様」で「JavaScriptが元」なのはJSONだけ。

UMLは「構造（形）」と「振る舞い（動き）」の図に分かれる



構造の図	表すもの
クラス図	クラスの属性・操作と関係（上の図）
オブジェクト図	ある時点の具体的なオブジェクトのつながり
コンポーネント図	ソフトウェアの部品とインタフェース
配置図	どのハードウェアに何を置くか

振る舞いの図	表すもの
ユースケース図	利用者（アクター）から見た機能
アクティビティ図	処理の流れ（分岐・並行）。流れ図に近い
ステートマシン図	1つのオブジェクトの状態の移り変わり
シーケンス図	メッセージのやり取りを**時間の順に上から下へ**
コミュニケーション図	オブジェクトの**つながり**を中心に、順番は**番号**で
タイミング図	横軸に時間をとり、状態が変わる時刻を正確に

> シーケンス図とコミュニケーション図は同じ内容を別の見せ方で表す。「時間の順」ならシーケンス図、「つながり」ならコミュニケーション図。

試験で狙われやすい比較10選（1/2）

	比較	見分ける一言
1	バブル vs 選択 vs 挿入ソート	**隣どうし**を交換／**最小値を探して**先頭と交換／整列済みの所へ**差し込む**
2	クイック vs マージソート	**基準値**で大小に分ける（最悪 O(n²)）／**半分ずつ**に分けて併合（いつも O(n log n)・安定）
3	スタック vs キュー	後入れ先出し（LIFO）／先入れ先出し（FIFO）
4	2分探索木 vs ヒープ	**左＜節＜右**（中間順で整列）／**親≧子**だけ（根が最大）
5	深さ優先 vs 幅優先探索	奥へ進んで戻る（スタック）／近い段から順に（キュー）

1・2・5・6・7 は、過去問で正解と誤りの選択肢として**同じ問題に並んだ**組合せ（R6春 問7、R3春 問6、R3秋 問6、R4春 問6 など）。似た言葉どうしを一言で見分けられるようにしておく。

試験で狙われやすい比較10選（2/2）

	比較	見分ける一言
6	再入可能 vs 再使用可能	同時に呼ばれても正しい（データはタスクごと）／終わったあと読み込み直さずに使える（始めに初期化）
7	再帰的 vs 再配置可能	自分自身を呼べる／主記憶のどこに置いても動く
8	オーバーロード vs オーバーライド	同じ名前で**引数が違う**／親のメソッドを**同じ形で書き直す**
9	静的型付け vs 動的型付け	実行前に型を決めて誤りを見つける（TypeScript・Scala）／実行時に値で決まる（Python・R）
10	DTD vs XML Schema、DOM vs SAX	独自の書き方／XMLの書き方でデータ型まで。全部読んで木にする／先頭から読んで都度処理

おまけ：ファースト（最初に入る）・ベスト（入る中で最小）・ワースト（最大）フィット、KMP（先頭から・自分の情報）・BM（末尾から）、値呼出し（コピー）・参照呼出し（場所）。

出典

過去問
IPA 応用情報技術者試験 午前 問題冊子・解答例（令和3年度春期〜令和7年度秋期の10回分）。この分野の問題（各回の問5〜問7、令和3年度春期は問5〜問8。計31問）を1問ずつ読み、正解と誤りの選択肢を確認した。IPAの過去問題は許諾・使用料なしで利用できる（IPA「過去問題」ページ）

シラバス
IPA「応用情報技術者試験（レベル3）シラバス Ver.7.2」中分類2 アルゴリズムとプログラミング（小分類1〜5の用語例）

イラスト
Microsoft Fluent Emoji（MIT License）。ライセンス全文は tools/slides/img/algo/LICENSE-fluentui-emoji.txt

イラスト（2）
いらすとや（https://www.irasutoya.com/）

作成
上の資料をもとにClaudeがまとめた。解説はIPAの公式解説ではなく、Claudeが自分の言葉で書いたもの。サイトの用語集（224語）と問題演習（43問）も同じ資料から作っている